MSA 및 Agile 방법론 학습

개인 서브노트

SKALA 4기 · Agile 방법론 및 MSA 개발

이름 김민정

번호 P282

소속 판교 캠퍼스

목차

Chapter 1. MSA 인프라 구성 요소 이해

1. 전체 구조와 기동 순서 / 2. 서비스 디스커버리(Eureka) / 3. API Gateway / 4. 인증 서버와 JWT / 5. 메시지 브로커(Kafka) / 6. 각 서비스의 Role & Functions

Chapter 2. API 명세 이해와 연동 확인

1. 라우팅 규칙과 실제 URL / 2. 서비스별 API 명세 / 3. 명세 확인 기록 / 4. FE↔BE 연동 성공 기록 / 5. 인증 동작 확인

Chapter 3. 요구사항 → 도메인 → 서비스 매핑

1. 요구사항 정리 / 2. 도메인 매핑표 / 3. 도메인 치환으로 확인한 서비스 경계

Chapter 4. Agile과 MSA의 결합 원리, 스프린트 분할 기준

1. 두 방법론이 맞물리는 지점 / 2. 스프린트를 나눈 기준 / 3. Sprint1·Sprint2 백로그

Chapter 5. AI 서비스에 MSA가 적합한가

1. AI 서비스의 네 가지 특성 / 2. 코드에서 확인한 근거 / 3. 모놀리식이었다면

Chapter 6. 아키텍처 선택 기준과 이해관계자 가치

1. 세 구조 비교 / 2. 판단 체크리스트 / 3. 실제 상황 적용 / 4. 이해관계자별 가치

부록. 궁금했던 점과 직접 알아본 것

1. 궁금했던 점 — 게이트웨이가 검증했는데 각 서비스도 또 검증하는 이유 / 2. 직접 알아본 것 — 소스가 없는 이미지의 설정을 바꾸는 방법 / 3. 트러블슈팅 기록 / 4. 마지막으로 느낀 점

Chapter 1. MSA 인프라 구성 요소 이해

1. 전체 구조와 기동 순서

```
MariaDB / Kafka (인프라)
  → Eureka (서비스 등록소)
    → Auth Server (인증)
      → API Gateway + 도메인 서비스 4종
        → Recommend Service
```

이 순서를 보고 나서 각 구성 요소의 성격을 구분했습니다. MariaDB와 Kafka는 데이터를 담는 그릇이고, Eureka와 Gateway와 Auth Server는 서비스가 아니라 **서비스를 둘러싼 인프라**이며, 실제 업무 로직은 user / course / enrollment / payment / recommend 다섯 개에만 들어 있습니다. 인프라 3종은 도메인을 몰라도 되고, 도메인 서비스 5종은 서로의 주소를 몰라도 됩니다.

구성 요소	포트	역할
MariaDB	3306	단일 인스턴스, 테이블 단위로 서비스별 데이터 분리
Kafka (KRaft)	9092	비동기 이벤트 메시지 버스
Eureka Server	8761	서비스 등록 / 탐색
Auth Server	9000	OAuth2 인증, JWT 발급, 공개키(JWK) 제공
API Gateway	8080	단일 진입점, 라우팅, 토큰 검증, 헤더 전파
도메인 서비스 5종	8081~8085	회원 · 상품 · 신청 · 결제 · 추천

2. 서비스 디스커버리 — Eureka

2-1. 왜 필요한지

모놀리식에서는 다른 기능을 부를 때 그냥 메서드를 호출하면 되므로 "주소"라는 개념 자체가 없습니다. 그런데 서비스가 쪼개지고 컨테이너로 뜨는 순간, 상대의 IP는 재시작할 때마다 바뀔 수 있고 인스턴스 개수도 달라집니다. 호출하는 쪽 코드에 IP를 적어 두면 상대가 재배포될 때마다 이쪽도 고쳐야 합니다. Eureka는 이 문제를 **주소 대신 이름으로 부르게 해서** 해결합니다.

2-2. 등록(Registration)과 조회(Discovery)

- **등록** — 각 서비스는 기동할 때 자기 이름(`spring.application.name`)과 호스트·포트를 Eureka에 올립니다. 그 뒤로는 30초 간격으로 하트비트를 보내 "살아 있다"고 알리고, 하트비트가 끊기면 Eureka가 목록에서 내립니다.
- **조회** — 부르는 쪽은 Eureka에서 서비스 이름으로 인스턴스 목록을 받아 그중 하나를 골라 호출합니다. Gateway의 `uri: lb://course-service`에서 `lb://`가 바로 "Eureka에서 이 이름을 찾아 로드밸런싱하라"는 뜻입니다.

```
eureka:
  client:
    register-with-eureka: false   # 자기 자신은 등록하지 않음
    fetch-registry: false         # 자기 자신은 조회하지 않음
  server:
    enable-self-preservation: false   # 개발 환경에서 self-preservation 비활성화
```

Eureka 서버도 `eureka.client` 설정을 갖는 이유는, Eureka가 원래 여러 대를 띄워 서로를 복제하도록 설계되어 있기 때문입니다. 실습은 한 대뿐이므로 자기 자신에게 등록·조회할 필요가 없어 둘 다 `false`로 둡니다. self-preservation은 하트비트가 갑자기 많이 끊겨도 "네트워크 문제일 것"이라고 보고 목록을 지우지 않는 보호 기능인데, 실습에서는 컨테이너를 자주 내렸다 올리므로 이미 죽은 서비스가 목록에 남는 게 오히려 혼란스러워서 꺼 둡니다.

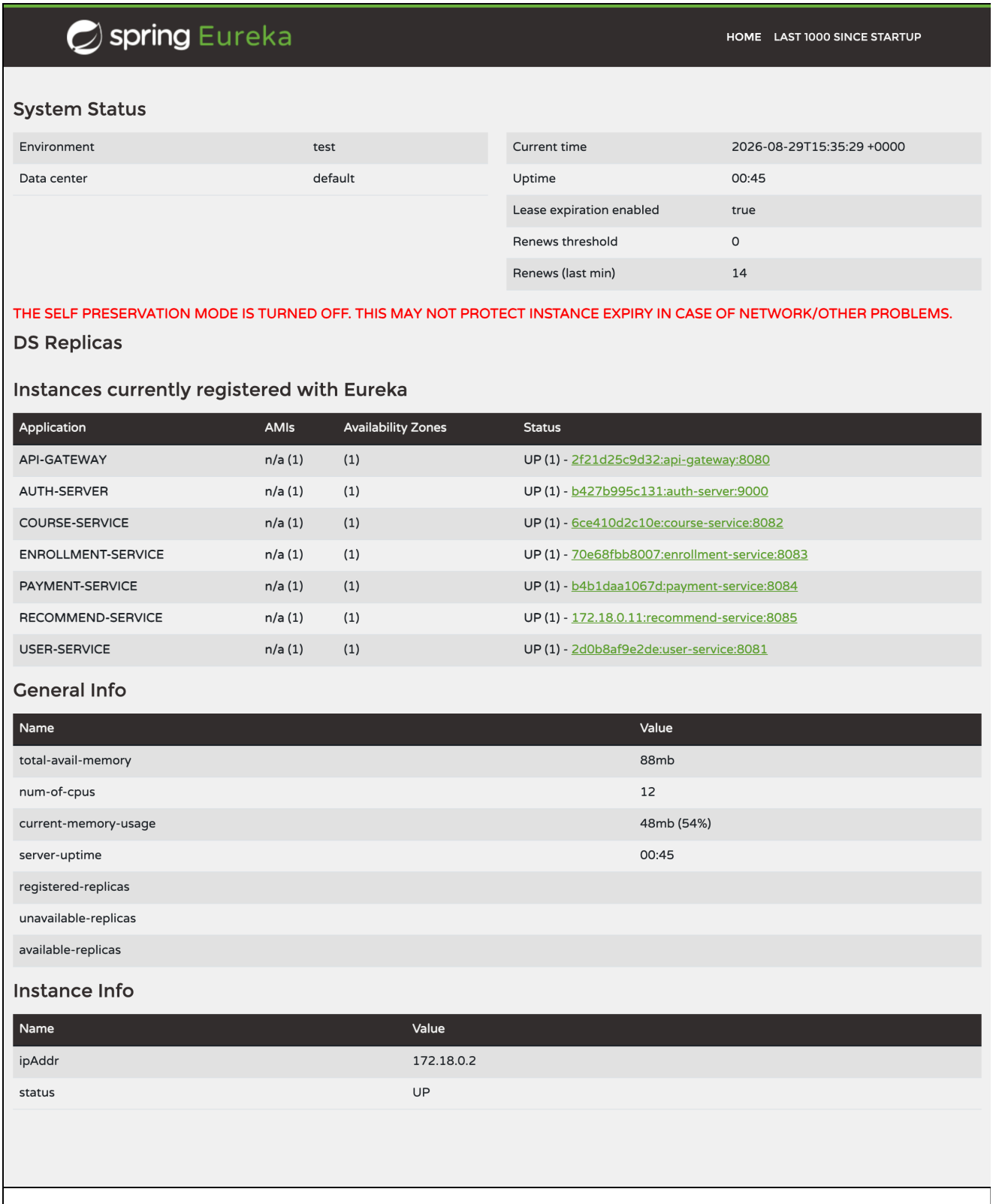
2-3. 등록 결과 확인

Eureka 대시보드(`http://localhost:8761`)에서 등록된 인스턴스를 눈으로 확인할 수 있고, 같은 내용을 REST로도 받을 수 있습니다. 실제로 조회했을 때 인프라와 도메인 서비스가 모두 `UP` 상태로 올라와 있는 것을 확인했습니다.

```
GET http://localhost:8761/eureka/apps   (Accept: application/json)

AUTH-SERVER         UP  :9000
API-GATEWAY         UP  :8080
USER-SERVICE        UP  :8081
COURSE-SERVICE      UP  :8082
ENROLLMENT-SERVICE  UP  :8083
PAYMENT-SERVICE     UP  :8084
RECOMMEND-SERVICE   UP  :8085
```

Gateway 자신도 Eureka에 **등록되는** 클라이언트이면서, 동시에 라우팅을 위해 목록을 **조회하는** 클라이언트이기도 합니다. 즉 Eureka는 게이트웨이 위에 있는 상위 존재가 아니라, 모두가 참조하는 전화번호부에 가깝습니다.



캡처 1. Eureka 대시보드(localhost:8761) — SK AM 스택의 인스턴스 등록 현황. API-GATEWAY·AUTH-SERVER·COURSE-SERVICE(상품)·ENROLLMENT-SERVICE(구독)·PAYMENT-SERVICE·RECOMMEND-SERVICE·USER-SERVICE 7개가 모두 UP 상태로 등록되어 있다.

3. API Gateway — 단일 진입점과 라우팅

3-1. 단일 진입점이라는 말의 의미

프런트엔드 코드를 보면 `axios`의 `baseURL`이 사실상 하나이고, 호출하는 경로가 `/api/users/...`, `/api/products/...`처럼 접두어만 다릅니다. 서비스가 다섯 개인데도 프런트가 아는 주소는 8080 하나뿐입니다. 이것이 단일 진입점입니다. 덕분에 프런트는 **백엔드가 몇 조각으로 나뉘어 있는지 알 필요가 없고**, CORS·인증·로깅 같은 공통 처리도 한 곳에서 처리됩니다.

3-2. 라우팅 규칙

경로 조건(Path)	대상(uri)	비고
`/oauth2/**, /login, /logout`	`lb://auth-server`	표준 프로토콜 경로
`/api/users/**`	`lb://user-service`	:8081
`/api/products/**`	`lb://course-service`	:8082
`/api/subscriptions/**`	`lb://enrollment-service`	:8083
`/api/payments/**`	`lb://payment-service`	:8084
`/api/recommend/**`	`lb://recommend-service`	:8085 (FastAPI)

라우트 하나는 **조건(predicates) + 대상(uri) + 가공(filters)** 세 부분으로 읽으면 구조를 파악할 수 있습니다. 예를 들어 "경로가 `/api/products/`로 시작하면(조건) → Eureka에서 course-service를 찾아 보내되(대상) → 경로를 `RewritePath`로 다시 써서 전달한다(가공)"입니다.

3-3. 인증 필터와 헤더 전파

필터는 `LoggingFilter(-2)` → `JwtAuthenticationFilter(-1)` 순서로 동작하며, JWT 필터는 토큰을 검증한 뒤 토큰 안의 클레임을 꺼내 아래 헤더로 바꿔 하위 서비스에 넘깁니다.

```
X-User-Id     : 사용자 식별자
X-User-Email  : 이메일
X-User-Role   : 권한 (BUYER / PROVIDER)
```

컨트롤러 코드를 보면 각 서비스가 토큰을 파싱하지 않고 헤더만 읽는 것을 알 수 있는데, 이렇게 설계했을 때 토큰 해석 로직을 다섯 서비스에 중복으로 넣지 않아도 된다는 장점이 있다고 생각했습니다.

```
@PostMapping
public ResponseEntity<...> subscribe(
        @Valid @RequestBody SubscriptionDto.SubscribeRequest request,
        @RequestHeader("X-User-Id") Long userId) { ... }
```

즉 토큰 해석이라는 공통 관심사를 게이트웨이가 한 번만 처리하고, 도메인 서비스는 "누가 요청했는지"만 `Long userId`로 받아 업무에 집중합니다. 다섯 서비스에 JWT 파싱 코드를 각각 넣지 않아도 되는 구조입니다. 이 흐름은 2장 3절의 Swagger 요청 캡처에서 `X-User-Id` 헤더가 실제로 어떻게 넘어가는지 다시 확인합니다.

4. 인증 서버 — JWT 발급과 검증 흐름

4-1. 발급까지의 흐름

지금까지 배운 JWT는 "로그인하면 서버가 토큰을 만들어 주고, 같은 서버가 그 토큰을 검증한다"였습니다. 여기서는 **발급자와 검증자가 분리**되어 있어서 처음에는 흐름이 낯설었습니다. 프런트 코드(`store/auth.js`, `api/auth.js`)를 따라가며 순서를 정리했습니다.

1. 사용자가 로그인 버튼을 누르면 프런트가 `/oauth2/authorize?response_type=code&client_id=...&redirect_uri=...`로 **이동**합니다. (Authorization Code Grant)
2. Auth Server(9000)의 로그인 화면에서 아이디·비밀번호를 확인하고, 성공하면 `redirect_uri`로 **인가 코드(code)**를 돌려줍니다.
3. 프런트의 콜백 화면이 그 코드를 받아 `POST /oauth2/token`으로 교환을 요청합니다. 이때 `client_id:client_secret`을 Base64로 인코딩해 Basic 헤더에 실어 보냅니다(CLIENT_SECRET_BASIC).
4. 응답으로 `access_token`(JWT)을 받아 `sessionStorage`에 저장하고, 이후 모든 요청에 `Authorization: Bearer ...`로 붙입니다.

비밀번호가 프런트를 거쳐 백엔드로 흘러가지 않고 Auth Server 화면에서만 입력된다는 점, 그리고 브라우저에 남는 것은 비밀번호가 아니라 만료 시간이 있는 토큰이라는 점이 이 방식의 핵심입니다.

4-2. 검증 흐름 — 공개키를 가져다 검증한다

토큰은 Auth Server의 RSA 개인키로 서명됩니다. 검증하는 쪽은 개인키가 없어도 되고, **공개키만** 있으면 서명이 맞는지 확인할 수 있습니다. 그래서 각 서비스 설정에 아래 항목이 들어 있습니다.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://auth-server:9000/oauth2/jwks
          issuer-uri:  http://auth-server:9000
```

FastAPI로 만들어진 recommend-service도 같은 일을 파이썬 코드로 직접 합니다. 이 파일을 읽고 검증이 구체적으로 어떤 동작인지 확인했습니다.

```
# recommend-service/app/config/security.py (요약)
jwks = await get_jwks()                       # ① 공개키 목록을 받아 캐시
signing_key = get_signing_key(token, jwks)    # ② 토큰 헤더의 kid로 키를 고름
payload = jwt.decode(token, signing_key,
                     algorithms=["RS256"],
                     issuer=settings.jwt_issuer_uri)   # ③ 서명·발급자·만료 검증
```

정리하면 검증은 ①공개키 확보 → ②kid로 서명키 선택 → ③서명·발급자(iss)·만료(exp) 확인의 세 단계입니다. 그리고 서비스 간 호출용으로는 사용자 토큰이 아니라 Client Credentials로 받은 토큰을 쓰며, scope에 service.read가 있는지까지 확인하는 함수(verify_service_token)가 따로 있습니다. 사용자 권한과 서비스 권한을 완전히 다른 축으로 관리하고 있었습니다.

구분	사용자 인증	서비스 간 인증
Grant Type	Authorization Code	Client Credentials
주체	사람(브라우저)	서비스 자신
검증 지점	Gateway → 클레임을 헤더로 전파	각 서비스가 scope 확인
대표 경로	`/api/subscriptions`	`/api/**/internal/**`

5. 메시지 브로커 — Kafka

5-1. 동기와 비동기를 나누는 기준

코드를 읽기 전에는 "서비스끼리는 REST로 부르면 되는데 왜 Kafka가 또 필요한가"가 가장 큰 의문이었습니다. 두 통신 방식이 쓰인 자리를 비교해 보고 기준을 이렇게 정리했습니다.

방식	쓰이는 자리	판단 기준
동기 REST (WebClient)	구독 신청 → 상품 존재 확인 구독 신청 → 결제 요청 추천 → 구독 이력 조회	**결과를 지금 당장 알아야** 다음 줄을 실행할 수 있는 경우. 상품이 없으면 애초에 구독을 만들면 안 되므로 기다려야 한다.
비동기 Kafka	`payment.completed` → 구독 활성화 `subscription.completed` → 추천 갱신	**알려 주기만 하면 되는** 경우. 결제가 끝났다는 사실만 던지고, 그 뒤 처리는 받는 쪽 사정에 맡긴다.

5-2. 실제 이벤트 흐름

```
[payment-service]  결제 COMPLETED
      │  publish  topic: payment.completed   key: userId
      ▼
[enrollment-service]  @KafkaListener 수신 → subscription PENDING → ACTIVE
      │  publish  topic: subscription.completed
      ▼
[recommend-service]  KafkaConsumer 수신 → 추천 캐시 갱신 트리거(실습에서는 로그)
```

결제 서비스는 구독 서비스를 **모릅니다**. 토픽에 던질 뿐입니다. 나중에 "결제가 끝나면 알림도 보내자"는 요구가 생기면 알림 서비스가 같은 토픽을 구독하면 되고, 결제 서비스 코드는 한 줄도 바뀌지 않습니다. 이것이 비동기를 쓰는 가장 큰 이유입니다.

5-3. 코드를 읽으며 확인한 세부 사항

- **메시지 키** — 두 producer 모두 `String.valueOf(event.getUserId())`를 키로 씁니다. 같은 사용자의 이벤트는 같은 파티션에 들어가므로 순서가 보장됩니다.
- **발행 실패 처리** — payment 쪽은 `.get(10, TimeUnit.SECONDS)`로 동기 대기하고 실패 시 예외를 던집니다. 반면 enrollment 쪽은 `whenComplete`로 로그만 남깁니다. 결제 완료 통보는 놓치면 구독이 영영 활성화되지 않지만, 추천 갱신 통보는 놓쳐도 서비스가 망가지지 않습니다. 그래서 두 쪽을 다르게 처리했습니다.
- **역직렬화** — consumer가 특정 DTO 대신 `Map<String, Object>`로 받습니다. 설정에도 `spring.json.use.type.headers: false`가 있습니다. 발행하는 쪽이 타입 헤더를 붙이지 않으므로, 받는 쪽이 특정 클래스를 기대하면 깨집니다. 서비스마다 패키지 구조가 다른 MSA에서는 **클래스가 아니라 필드 이름이 계약**입니다.
- **토픽 생성** — `KafkaConfig`에서 `NewTopic` 빈으로 파티션 3·복제 1의 토픽을 선언해, 브로커에 없으면 애플리케이션 기동 시 만들어 줍니다.

6. 각 서비스의 Role & Functions

서비스	스택 / 포트	Role (책임)	Functions (주요 기능)
User Service	Spring Boot 8081	회원 정보의 단일 소유자	회원가입, 내 정보 조회, 내부 조회용 API
Course Service (상품 카탈로그)	Spring Boot 8082	판매 대상(상품/강의) 카탈로그 관리	등록, 목록·상세·카테고리 조회, 존재 확인, 구독자 수 증가, 추천용 후보 조회
Enrollment Service (구독/신청)	Spring Boot 8083	신청 상태의 수명주기 관리 (PENDING→ACTIVE→CANCELLED)	신청, 내 목록 조회, 이력 조회, 결제 이벤트 수신 후 활성화, 완료 이벤트 발행
Payment Service	Spring Boot 8084	결제 트랜잭션 기록과 완료 통보	내부 결제 요청 처리, 결제 내역 조회, `payment.completed` 발행
Recommend Service	FastAPI 8085	이력 기반 규칙 추천 (읽기 전용)	사용자별 추천 목록 반환, 이벤트 수신, 신규 사용자 인기순 폴백

표로 정리하면서 **테이블 하나에 주인 서비스가 하나**라는 점을 확인할 수 있었습니다. 구독 서비스는 상품 정보가 필요할 때 `api_products` 테이블을 직접 조회하지 않고 course-service의 내부 API를 호출합니다. DB는 물리적으로 한 대(MariaDB)를 쓰지만 접근 경로를 서비스로 제한해서, 나중에 DB를 진짜로 분리하더라도 코드는 바뀌지 않습니다.

Chapter 2. API 명세 이해와 연동 확인

1. 라우팅 규칙과 실제 URL

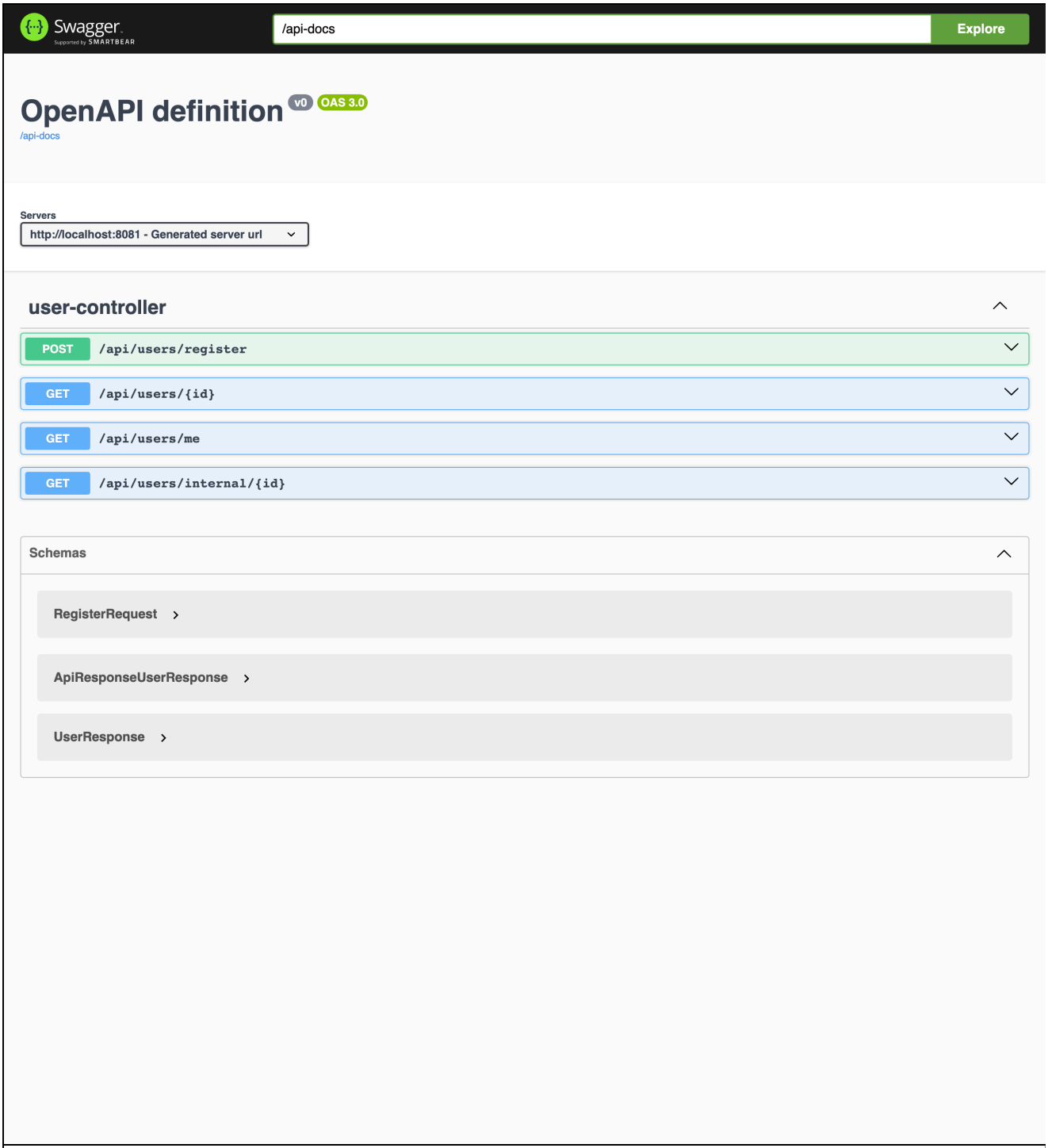
프런트가 호출하는 URL, 게이트웨이가 판단하는 경로, 서비스 컨트롤러의 매핑이 어떻게 이어지는지 한 줄로 따라가 보면 명세가 한눈에 들어옵니다.

```
브라우저          http://localhost:3000  →  vite proxy( /api → :8080 )
게이트웨이        Path=/api/subscriptions/**  →  lb://enrollment-service
서비스 컨트롤러   @RequestMapping("/api/subscriptions")  @PostMapping
```

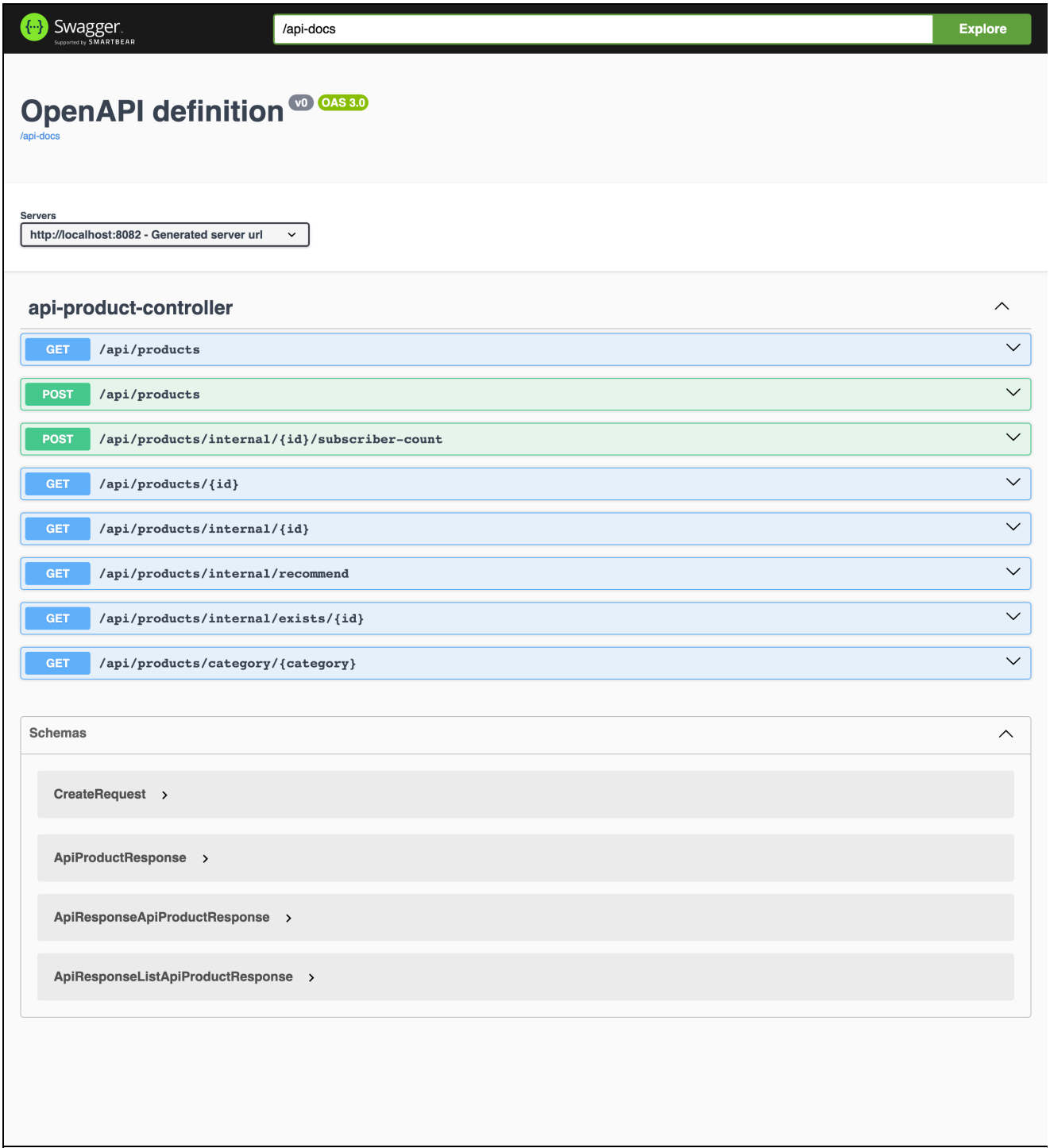
개발 중에는 vite의 `server.proxy`가 `/api`·`/oauth2`·`/login` 요청을 8080으로 넘겨 주기 때문에, 프런트에서는 도메인 없이 `/api/...`만 적어도 게이트웨이까지 도달합니다. 이 설정을 보기 전에는 프런트가 8080을 직접 부르는 줄 알았는데, 실제로는 프록시를 한 번 거치고 있었습니다.

2. 서비스별 API 명세

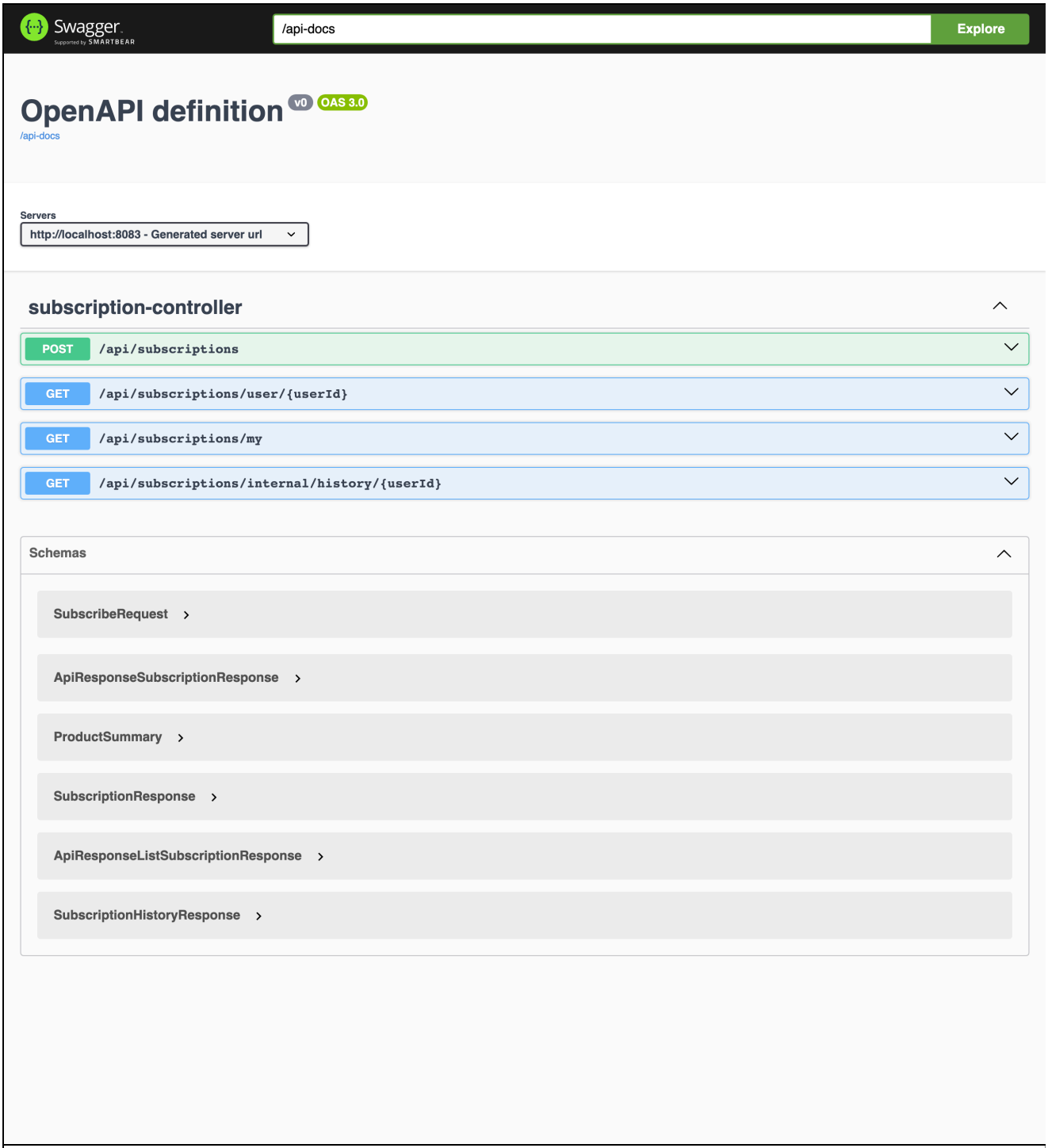
명세는 표로 옮겨 적는 대신, 각 서비스의 Swagger UI(springdoc)를 직접 열어 캡처했습니다. 게이트웨이(8080) 뒤에 있는 4개 서비스 각각이 `/swagger-ui/index.html`을 띄우고 있어서, 포트만 바꿔 가며 확인할 수 있었습니다.



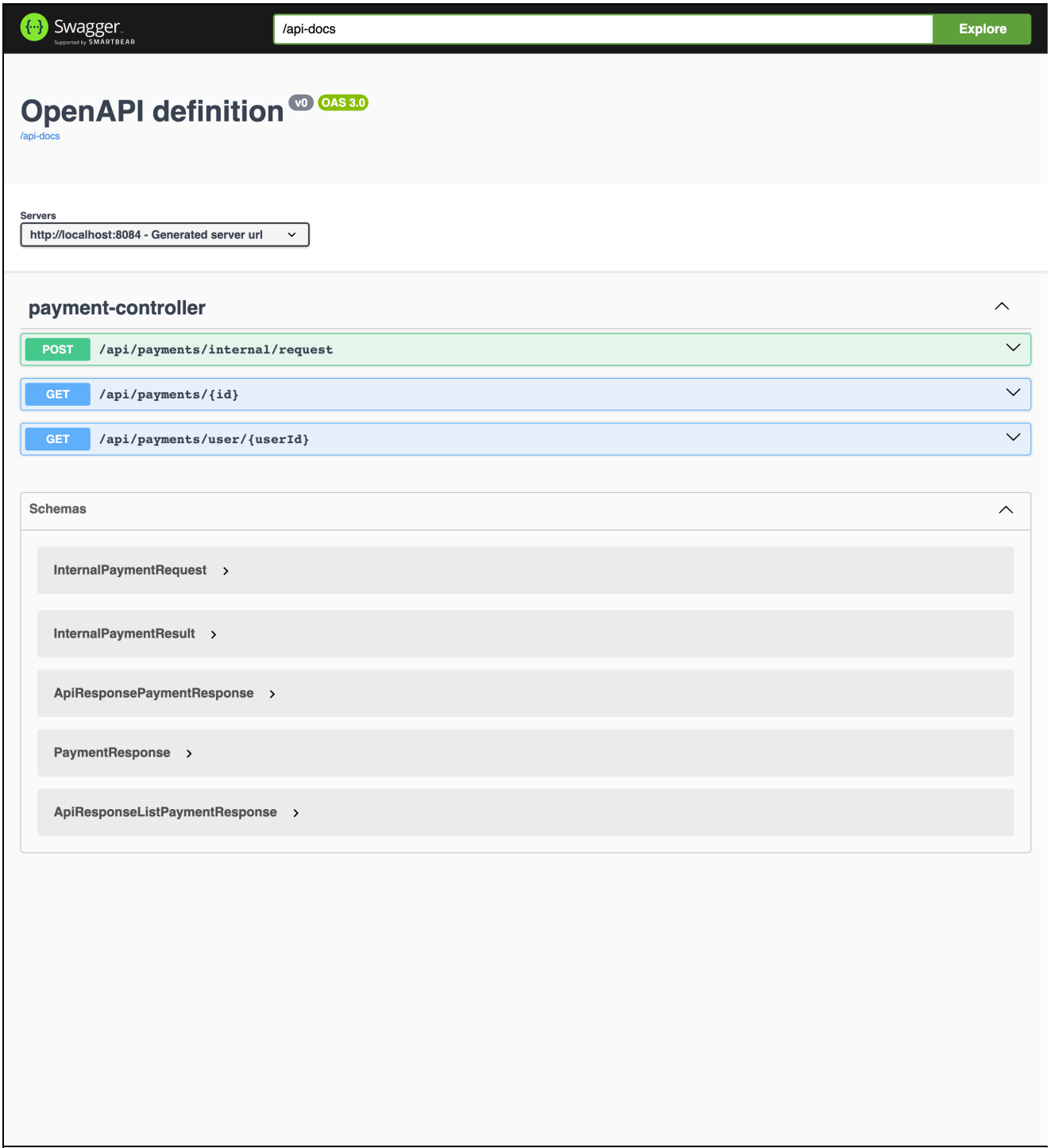
캡처 2-1. User Service(:8081) Swagger — `user-controller`. 회원가입·내 정보 조회·내부 조회용 엔드포인트 4개.



캡처 2-2. Course Service(:8082, 문서상 명칭은 Product Service) Swagger — `api-product-controller`. 등록·조회·카테고리·내부 호출용 8개 엔드포인트.



캡처 2-3. Enrollment Service(:8083, 문서상 명칭은 Subscription Service) Swagger — `subscription-controller`. 구독 신청·내 구독·이력 조회 4개 엔드포인트.

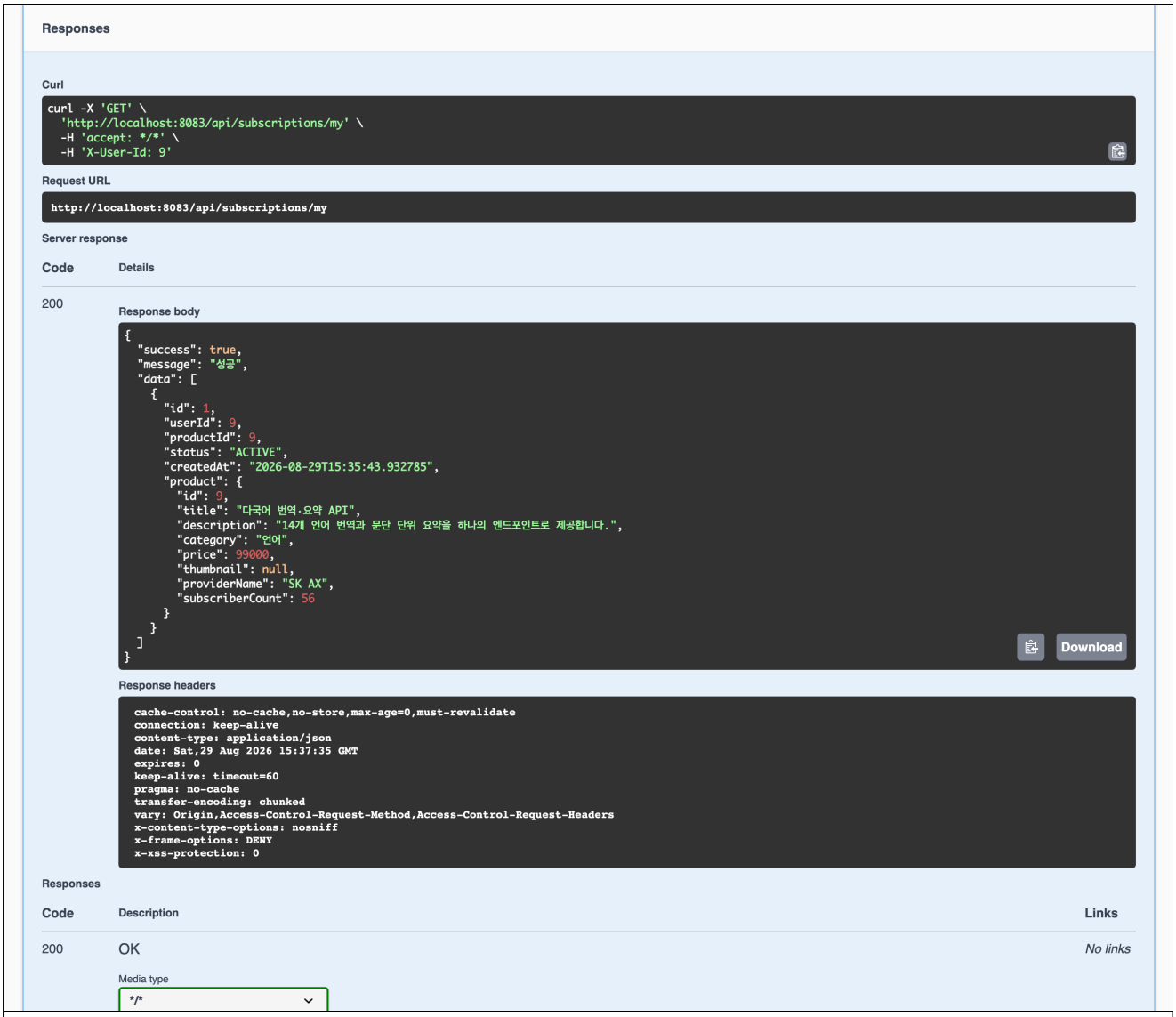


캡처 2-4. Payment Service(:8084) Swagger — `payment-controller`. 내부 결제 요청·결제 조회 3개 엔드포인트.

명세를 옮겨 적으면서 눈에 띈 것은 경로에 들어 있는 `internal`이었습니다. `/api/products/internal/exists/{id}`, `/api/subscriptions/internal/history/{userId}`처럼 서비스끼리만 쓰는 엔드포인트를 경로 이름으로 구분해 둡니다. 응답 형태도 다릅니다. 화면용 API는 `{success, message, data}` 래퍼로 감싸지만, 내부용 API는 래퍼 없이 값만 그대로 돌려줍니다. 부르는 쪽이 사람이 아니라 코드이므로 굳이 메시지를 붙일 이유가 없기 때문입니다.

3. 명세 확인 기록

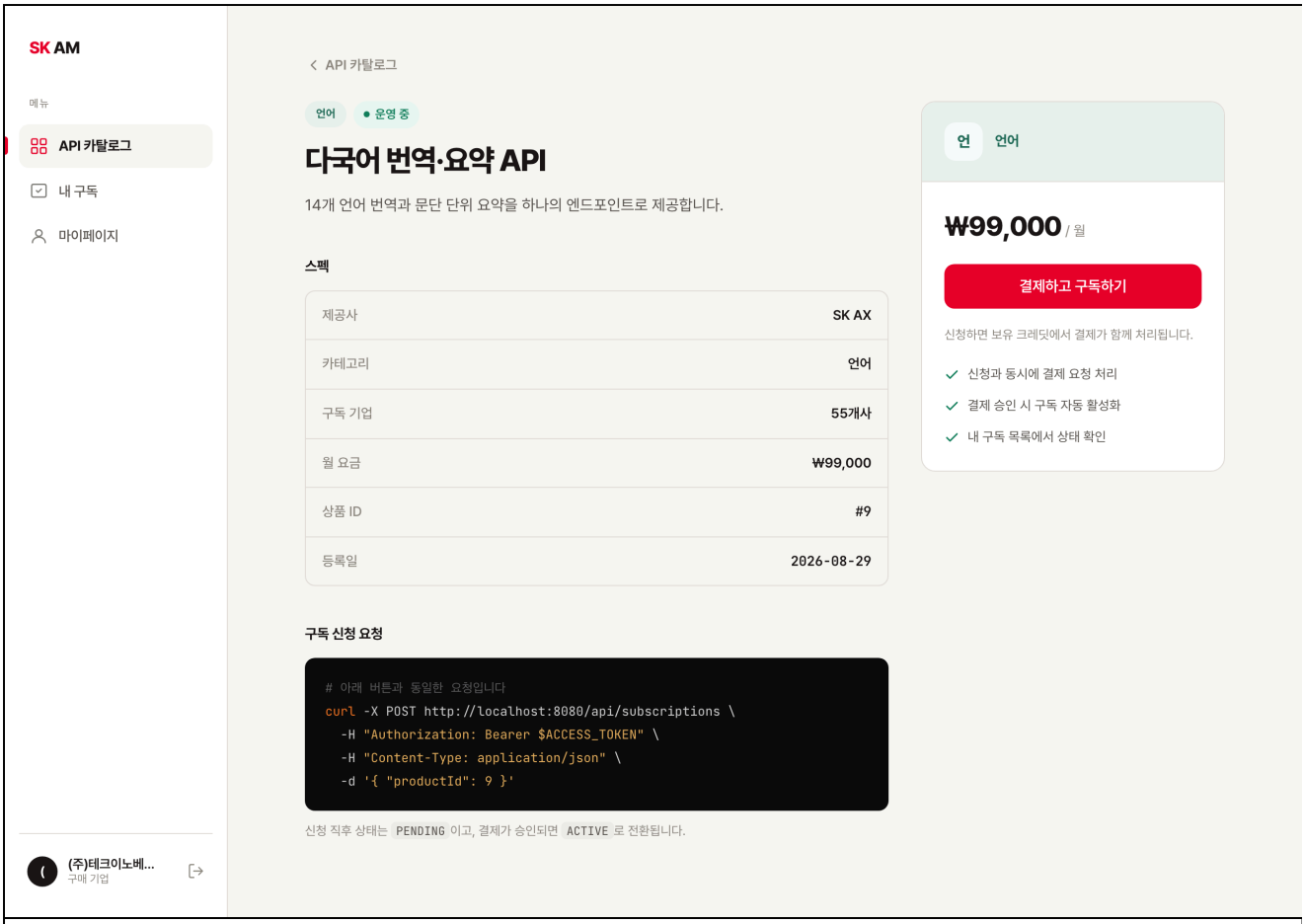
화면 대신 Swagger UI에서 직접 엔드포인트를 열어 **Try it out**으로 실제 요청을 보내고 응답을 확인했습니다. 아래는 Subscription Service의 `GET /api/subscriptions/my`를 `X-User-Id: 9` 헤더로 호출한 결과입니다.



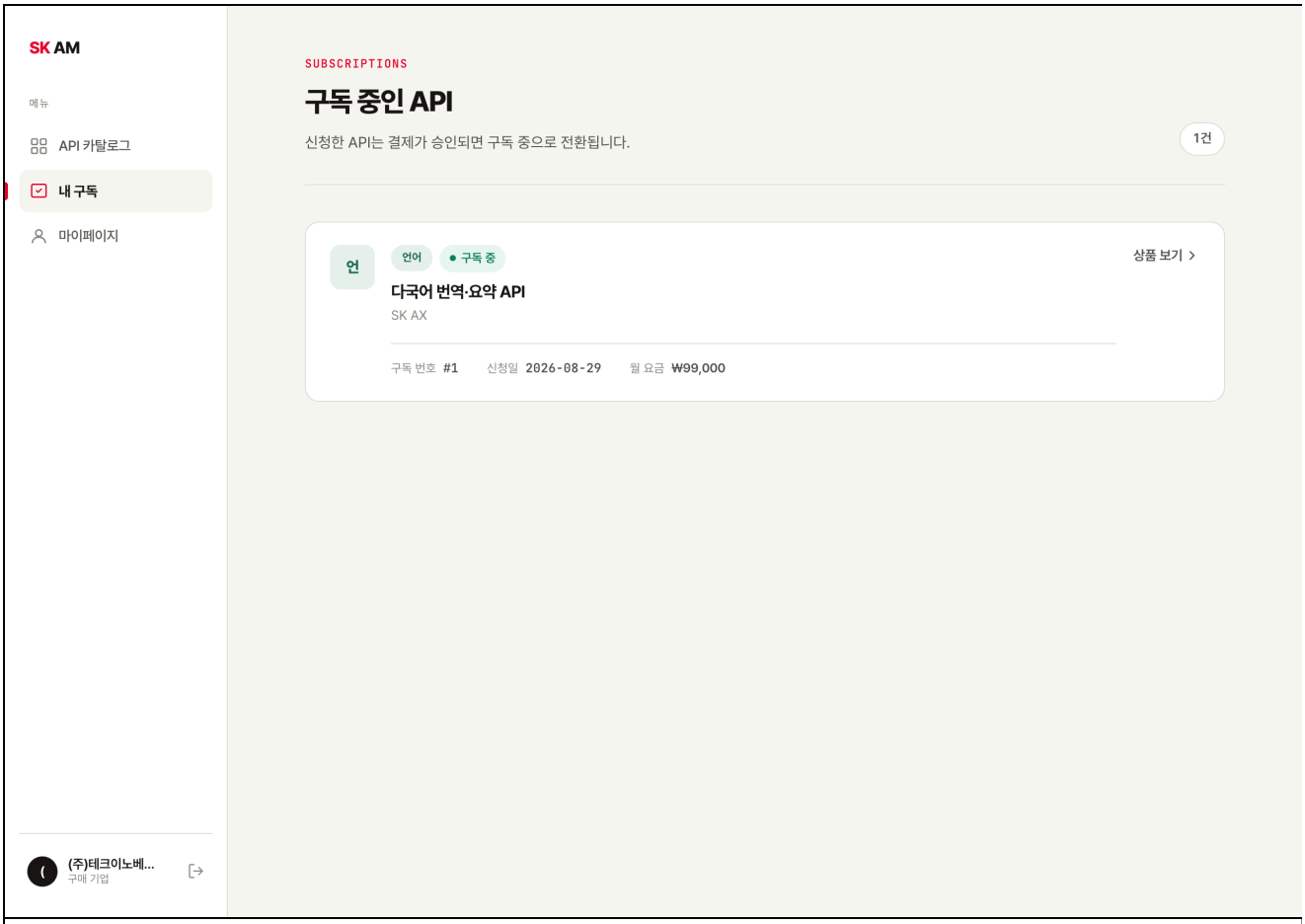
캡처 3. Swagger UI에서 실행한 curl 요청과 실제 응답. 신청한 상품이 `status: "ACTIVE"`로 조회되고, product 정보(제공사·구독 기업 수 포함)가 함께 붙어 온다.

4. FE↔BE 연동 성공 기록

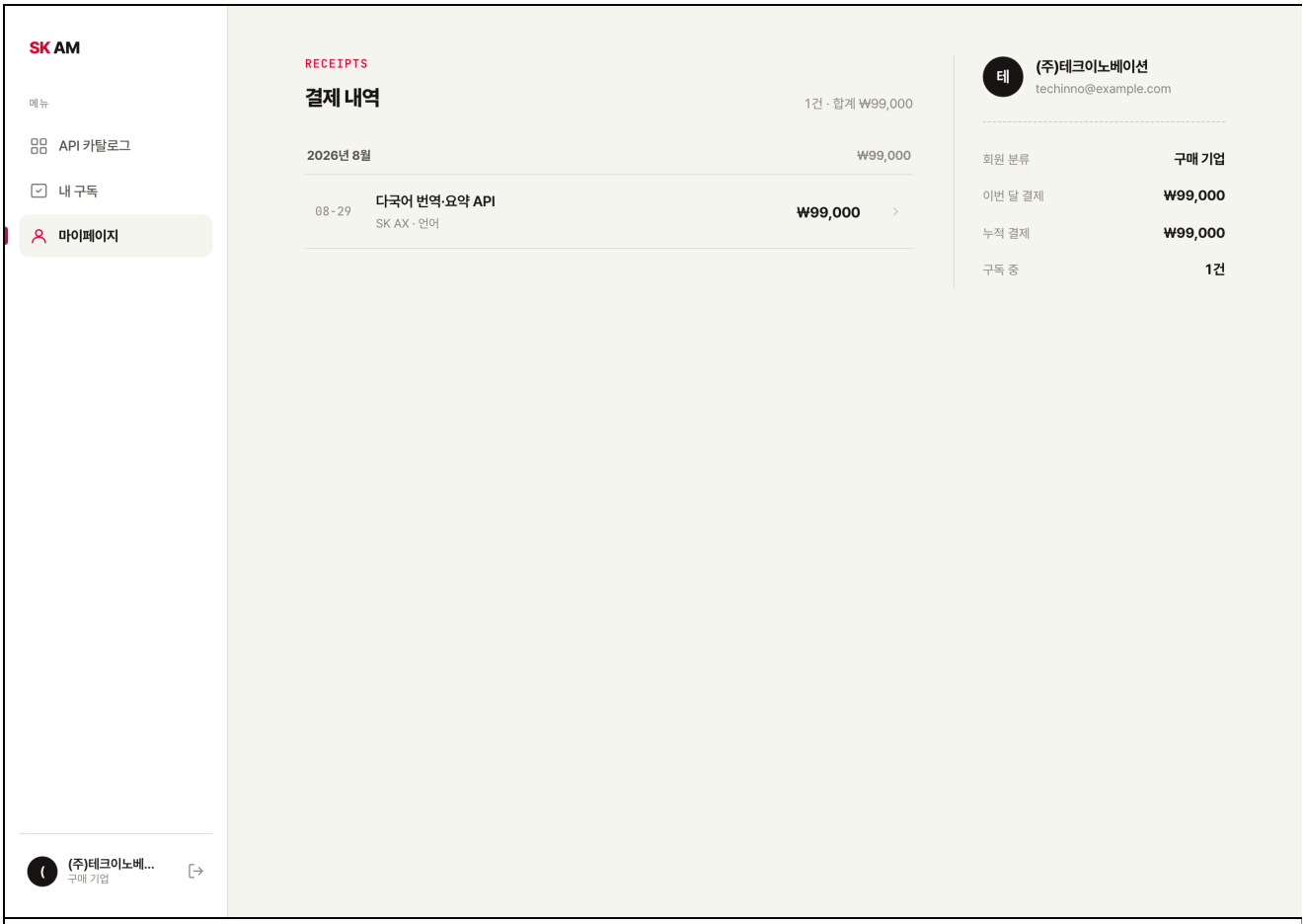
한 번의 클릭이 여러 서비스를 거쳐 화면으로 돌아오는 과정을 눈으로 확인하기 위해, 신청 전과 후의 화면을 함께 남겼습니다.



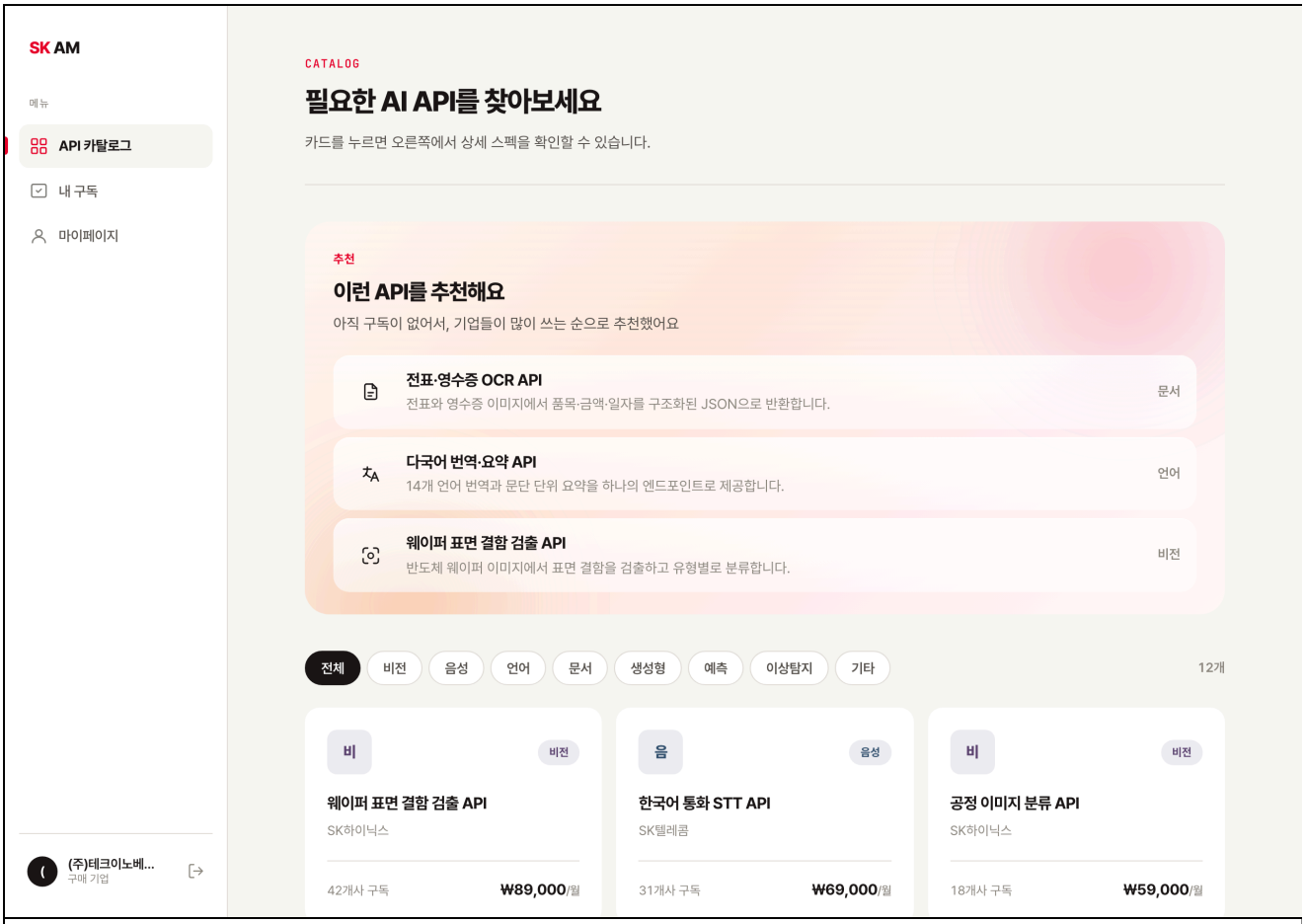
캡처 4. Before — 다국어 번역·요약 API 상세 화면. 아직 구독하지 않은 상태라 "결제하고 구독하기" 버튼이 활성화되어 있다.



캡처 5. After ① — 내 구독 목록. Kafka 이벤트(payment.completed → subscription.completed)를 거쳐 상태가 "구독 중"(ACTIVE)으로 바뀐 것을 확인할 수 있다.



캡처 6. After ② — 마이페이지 결제 내역. 같은 구독 건이 ₩99,000 결제 완료(COMPLETED) 건으로 함께 조회된다.



캡처 7. 추천 영역 — API 카탈로그 상단의 "이런 API를 추천해요" 배너. 구독 이력이 없는 신규 사용자에게는 구독 기업 수 기준 인기 API가 노출된다.

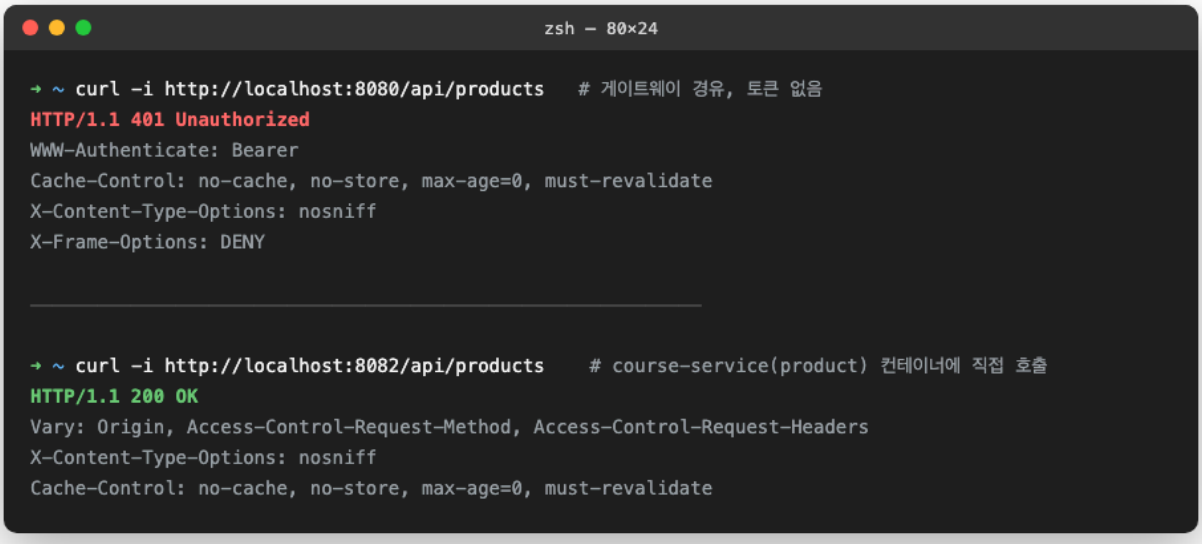
화면에서 버튼 한 번을 눌렀을 때 뒤에서 일어나는 일을 순서대로 적으면 아래와 같습니다.

```
① 브라우저   POST /api/subscriptions            (Bearer 토큰 첨부)
② 게이트웨이  토큰 검증 → X-User-Id 헤더로 변환 → enrollment-service
③ enrollment → course-service  GET /internal/exists/{id}   (동기, 존재 확인)
④ enrollment PENDING 저장 (독립 트랜잭션으로 먼저 커밋)
⑤ enrollment → payment-service POST /internal/request      (동기, 결제)
⑥ payment    결제 COMPLETED 저장 → payment.completed 발행   (비동기)
⑦ enrollment 이벤트 수신 → ACTIVE 변경 → 구독자 수 증가 →
             subscription.completed 발행
⑧ 브라우저   GET /api/subscriptions/my → 화면에 ACTIVE 표시
```

> ④에서 `SubscriptionWriteService.createPendingSubscription()`이 `@Transactional(propagation = REQUIRES_NEW)`로 선언된 이유를 한참 들여다봤습니다. 결제 이벤트(⑦)가 아주 빠르게 돌아오는데, 신청 레코드가 아직 커밋되지 않은 상태라면 이벤트를 받은 쪽이 `findByUserIdAndProductId`로 그 레코드를 찾지 못합니다. 그래서 PENDING 저장만 별도 트랜잭션으로 **먼저 커밋**해 두는 것입니다. 동기 호출과 비동기 이벤트가 섞이면 DB 커밋 시점까지 함께 따져야 합니다.

5. 인증 동작 확인

게이트웨이가 정말로 토큰을 검사하는지 확인하려고, 토큰 없이 같은 경로를 호출해 보았습니다.



캡처 8. 같은 `/api/products` 경로를 게이트웨이(8080)로 호출하면 401, course-service 컨테이너(8082)에 직접 호출하면 200이 돌아온다.

게이트웨이를 통하면 401이지만, 컨테이너 포트로 직접 호출하면 응답이 옵니다. 실습 환경에서는 각 서비스의 `SecurityConfig`가 `anyRequest().permitAll()`로 열려 있고, 원래의 Resource Server 설정은 주석으로 남아 있기 때문입니다. 이 부분을 보고 **"게이트웨이만 막으면 안전한가"**라는 의문이 생겼는데, 이는 부록에 정리했습니다.

Chapter 3. 요구사항 → 도메인 → 서비스 매핑

1. 요구사항 정리

서비스 경계를 어떻게 잡았는지 이해하려면 코드보다 팀이 정한 서비스 기획을 먼저 봐야 했습니다. 저희 팀(4조 블루베리스무디)이 정한 서비스는 **SK AM(SK API Marketplace)**으로, SK 계열사·파트너사가 만든 AI API를 구매 기업이 월 정액 구독 형태로 조달하는 B2B 마켓플레이스입니다. 시장 조사 단계에서 확인한 배경은 두 가지였습니다. 계열사별 AI 자산(SKT LLM, SK하이닉스 비전 등)은 검증되어 있지만 정부사업·지원 프로그램 단위로만 제공되어 상시 연결되지 않고 있었고, AI API 시장 자체는 연 32%씩 성장하고 있었습니다. 검증된 공급과 빠르게 늘어나는 수요 사이를 잇는 창구가 없다는 것이 출발점이었습니다.

이해관계자별 Pain Point를 정리하면 아래와 같습니다.

이해관계자	Pain Point	SK AM의 해결 방향
구매 기업 (Consumer)	AX 솔루션을 통째로 도입하기엔 초기 비용·러닝커브가 부담되고, 종량제는 과금 예측이 어렵다	필요한 API만 월 단위로 구독하는 정액제. 초기 CAPEX 없이 OPEX로 시작
API 공급사 (Producer)	대형 엔터프라이즈 B2B 계약에 편중되어 있어 소규모·다품종 수요는 놓치고 매출이 들쭉날쭉하다	다수 중소기업의 월 구독료가 모이는 안정적 반복 매출(MRR) 구조
플랫폼 (SK AX)	계열사마다 API 품질·인증·과금 방식이 제각각이라, 수요 기업 입장에서 신뢰할 통합 창구가 없다	SK AX가 인증·구독 결제·SLA를 표준화해 단일 신뢰 플랫폼으로 제공

2. 도메인 매핑표

템플릿(온라인 교육 플랫폼)의 구조를 그대로 재활용하고 개념만 SK AM 도메인으로 치환하는 방식으로 설계했습니다. 팀 기획서에 정리한 매핑은 아래와 같습니다.

템플릿 개념	SK AM 개념
강사 / 학생	API Vendor(API 공급사) / 구매 기업 담당자
과목 등록	API 상품 등록 (카테고리, 요금, 스펙)
과목 조회	API 카탈로그 탐색
수강신청	API 구독 신청
결제	구독 결제
수강권한 노출	구독 상태(ACTIVE)로 API 호출 권한 표현
수강생 수	구독 기업 수 (인기도 지표)
추천	사용 패턴 기반 API 추천

이 개념 매핑을 실제 서비스·테이블 단위로 다시 펼치면 다음과 같습니다.

요구사항(기능)	도메인	서비스	소유 테이블	대표 API
회원가입 · 로그인 · 내 정보	회원	User Service	`users`	`POST /api/users/register`
API 상품 등록 · 탐색 · 카테고리	카탈로그	Course Service (문서상 Product Service)	`api_products`	`GET /api/products`
구독 신청 · 상태 관리	구독	Enrollment Service (문서상 Subscription Service)	`subscriptions`	`POST /api/subscriptions`
구독 결제 처리 · 결제 내역	결제	Payment Service	`payments`	`POST /api/payments/internal/request`
사용 패턴 기반 API 추천	추천	Recommend Service	없음 (읽기 전용)	`GET /api/recommend/{userId}`
결제 완료 시 자동 활성화	도메인 간 연결	Kafka 이벤트	-	`payment.completed`
인증 · 토큰 발급	인프라	Auth Server	`users` 참조	`POST /oauth2/token`
단일 진입 · 라우팅	인프라	API Gateway	-	-
서비스 등록 · 탐색	인프라	Eureka Server	-	-

매핑을 하면서 세운 기준은 세 가지였습니다. 첫째, **데이터 소유권** — 하나의 테이블은 하나의 서비스만 쓴다. 둘째, **변경 이유** — 카탈로그 정책이 바뀔 때 결제 코드가 함께 바뀌면 안 된다. 셋째, **주체** — API 공급사가 쓰는 기능과 구매 기업 담당자가 쓰는 기능은 사용 빈도와 성격이 다르다. 추천을 별도 서비스로 둔 이유도 같은 기준에서 나옵니다. 추천은 자기 테이블이 없고 남의 데이터를 읽기만 하는데, 규칙이 자주 바뀌고 실패해도 서비스 전체가 멈추면 안 되는 성격이므로 따로 두는 편이 낫습니다.

3. 도메인 치환으로 확인한 서비스 경계

도메인을 통째로 바꿔 보면서 원래 서비스 경계가 어디까지 유지되고 어디부터 새로 손대야 하는지 확인했습니다. 팀 Task 분해표를 코드 기준으로 다시 정리하면 다음과 같습니다.

원본(교육 플랫폼)	치환(SK AM)	바뀐 것 / 바뀌지 않은 것
강의(Course)	API 상품 (ApiProduct)	엔티티·경로(`/api/products`)와 카테고리 8종 (VISION/SPEECH/LANGUAGE/DOCUMENT/GENERATIVE/FORECAST/ANOMALY/OTHER)은 바뀌었지만, 컨테이너명과 Eureka 등록명은 `course-service` 그대로
수강신청 (Enrollment)	구독 (Subscription)	상태값 PENDING/ACTIVE 구조와 `courseId → productId` 필드 치환. 컨테이너명은 `enrollment-service` 유지
수강생 / 강사	구매 기업 담당자(BUYER) / API 공급사 (PROVIDER)	DB 컬럼값은 STUDENT/INSTRUCTOR로 유지하고 `RoleConverter`로 변환
결제 · 추천	구독 결제 · 사용 패턴 기반 추천	필드명(`courseId → productId`)만 치환, 로직 구조는 그대로 통과

이 작업에서 신경 쓴 것은 두 가지입니다. 하나는 **이름을 바꿀 범위와 바꾸지 않을 범위를 가르는 일**이었습니다. Eureka 등록명이나 컨테이너명까지 한꺼번에 바꾸면 게이트웨이 라우팅, compose 의존성, 서비스 간 URL이 모두 연쇄로 깨집니다. 그래서 **겉으로 드러나는 경로·엔티티·화면 문구만 바꾸고, 서비스 식별자는 유지**하는 선택을 했습니다.

다른 하나는 `RoleConverter` 사례입니다. 인증 서버는 소스가 없는 이미지로 배포되어 있고 내부적으로 `users.role`을 STUDENT/INSTRUCTOR로 읽습니다. 컬럼값을 BUYER로 바꾸는 순간 인증 서버의 enum 파싱이 실패해 로그인이 깨집니다. 그래서 **물리 저장값은 레거시로 고정하고, 애플리케이션 안에서만 새 이름으로 변환**하는 방식을 택했습니다. 내가 고칠 수 없는 남의 서비스와 계약이 걸려 있을 때 어디까지 바꿀 수 있는지를 판단해야 한다는 것이, MSA에서는 실제로 벌어지는 상황입니다.

Chapter 4. Agile과 MSA의 결합 원리, 스프린트 분할 기준

1. 두 방법론이 맞물리는 지점

Agile은 개발 방식이고 MSA는 시스템 구조라서 원래 별개인데, 실습을 해 보니 둘은 서로를 필요로 하는 관계였습니다.

Agile이 요구하는 것	MSA가 제공하는 것
짧은 주기마다 동작하는 결과물(Increment)	서비스 단위 독립 배포. 전체를 다시 빌드하지 않고 한 조각만 올릴 수 있다.
스프린트마다 범위를 바꿔도 되는 유연함	경계가 나뉘어 있어 변경 영향 범위가 서비스 안으로 제한된다.
팀이 독립적으로 일할 수 있을 것	서비스=담당 디렉터리이므로 충돌 없이 병렬 작업이 가능하다.
매 스프린트 리뷰에서 데모	API 계약만 맞으면 미완성 서비스가 있어도 흐름 시연이 가능하다.

반대 방향도 성립합니다. MSA는 조각이 많아 한 번에 다 만들면 통합에서 무너지기 쉬운데, Agile의 짧은 반복이 **"조금 만들고 바로 통합해 본다"**는 안전장치가 되어 줍니다. 실제로 실습에서도 서비스 다섯 개를 동시에 조금씩 건드리는 방식이 아니라, 흐름 하나를 끝까지 통과시키는 방식으로 진행하도록 되어 있었습니다.

2. 스프린트를 나눈 기준

저희 팀의 스프린트 분할 기준을 한 문장으로 정리하면 **"최소한의 MVP를 먼저 뽑았다"**입니다. 기능을 균등하게 절반씩 나누는 대신, "SK AM이 최소한 이 목적 하나 — 필요한 API를 찾아서 구독 신청까지 끝내는 것 — 을 완결된 상태로 수행하는가"를 기준으로 Sprint1의 범위를 정했습니다. 여기 해당하는 기능은 다 돌아가도록 만들고, 그 바깥에 있는 "있으면 더 좋은" 기능은 확장으로 분류해 Sprint2로 미뤘습니다.

구체적으로는 세 가지를 기준으로 각 User Story를 Sprint1과 Sprint2로 나눴습니다.

1. **핵심 가치를 증명하는가** — 회원가입·로그인부터 API 카탈로그 조회·등록·구독 신청까지는 서비스의 가치 제안(필요한 API를 찾아 구독한다)을 그 자체로 증명하는 흐름입니다. Product Backlog에서도 이 항목들의 우선순위가 대부분 High였습니다.
2. **없어도 핵심 흐름이 끊기지 않는가** — 카테고리 필터는 검색을 더 편하게 해 줄 뿐 구독 자체를 막지 않고, 마이페이지의 추천 노출과 결제 화면도 이미 Sprint1에서 만든 구독 흐름 위에 얹는 부가 가치입니다. 이런 기능은 Sprint2로 미뤘습니다.
3. **선후 관계상 먼저 있어야 하는가** — 상품이 등록되어 있어야 구독을 신청할 수 있으므로, API 등록·조회가 구독보다 먼저 와야 합니다. 데이터가 만들어지는 순서가 곧 개발 순서였습니다.

3. Sprint1 · Sprint2 백로그

구분	목표	범위	분할 근거
Sprint 1 (MVP)	회원가입부터 구독 신청까지 핵심 흐름 한 바퀴가 끝까지 동작 (2일차 오전~13시)	로그인·로그아웃·회원가입 / API 조회 및 등록 / 구독 신청 / 내 구독 확인 — User Story US-01~07, 계획 24 SP 전량 소진(달성률 100%)	이 네 가지만 갖춰져도 "필요한 API를 찾아서 구독한다"는 서비스의 핵심 가치는 완결된 상태로 증명됩니다. 부가 기능을 걷어내고 이 흐름부터 확실히 통과시키는 데 시간을 썼습니다.
Sprint 2 (확장)	구독·탐색 경험을 확장하고, 결제·추천까지 이어짐 (2일차 13~15시)	카테고리 필터 / 구독 월정액 결제 / 마이페이지(추천 확인) — User Story US-08~12, 계획 18 SP 중 13 SP 소진 (달성률 72%, US-09 마이페이지 추천 5점은 다음 스프린트로 이월)	Sprint1에서 검증된 흐름 위에 얹는 부가 가치 기능들입니다. 시간이 부족했을 때 가장 먼저 미룰 수 있는 항목이기도 해서, 실제로 추천 노출 한 항목이 이월되었습니다.

막상 나눠 보니 **"이번 스프린트에서 무엇을 안 할지"**를 정하는 쪽이 더 중요했습니다. 결제 화면과 추천을 Sprint1에서 손대지 않기로 정했기 때문에, 남은 시간을 회원가입·등록·구독 흐름을 확실히 통과시키는 데 쓸 수 있었고, 실제로 Sprint1은 계획한 24 SP를 전량 소진해 100% 달성했습니다.

Chapter 5. AI 서비스에 MSA가 적합한가

1. AI 서비스의 네 가지 특성

"AI 서비스니까 MSA"라고 외우는 대신, AI 기능이 일반 CRUD 기능과 무엇이 다른지부터 정리하고 그 차이가 구조로 이어지는지 확인해 보았습니다.

AI 서비스의 특성	그래서 생기는 요구	MSA가 답이 되는 방식
반복 학습과 모델 개선 모델·추천 규칙은 자주 바뀐다	다른 기능은 그대로 둔 채 추천 로직만 자주 바꾸고 되돌리고 싶다	추천 서비스만 재배포. 회원·결제는 재시작조차 하지 않는다. A/B 비교나 롤백 단위도 서비스 하나로 좁혀진다.
부하 특성이 다름 추론은 CPU·메모리를 많이 쓰고 응답이 느릴 수 있다	무거운 추론이 로그인·결제 응답을 잡아먹지 않아야 한다	부하 격리. 추천만 인스턴스를 늘리거나 자원을 더 주면 되고, 느려져도 다른 서비스의 스레드를 점유하지 않는다.
기술 스택이 다름 모델·데이터 처리 생태계는 파이썬 중심	업무 로직은 Spring, 추론은 Python으로 쓰고 싶다	서비스별 스택 선택 자유. 실제로 추천만 FastAPI이고 나머지는 Spring Boot다.
결과가 항상 정확하지 않아도 됨 추천이 비어도 거래는 성립한다	AI 기능 장애가 핵심 거래를 막으면 안 된다	장애 격리와 우아한 성능 저하(graceful degradation). 추천이 죽어도 신청·결제는 그대로 동작한다.

2. 코드에서 확인한 근거

위 네 가지가 말뿐이 아니라 실제 코드에 나타나 있는지 확인했습니다.

- **다른 스택** — recommend-service만 `main.py` 기반 FastAPI이고, Eureka 등록도 `py_eureka_client`로 직접 합니다. Spring이 아니어도 같은 등록소·같은 게이트웨이 아래 들어올 수 있다는 것이 서비스 사이의 계약은 언어가 아니라 HTTP·JWT·Kafka입니다.
- **실패해도 죽지 않음** — 구독 이력 조회가 실패하면 예외를 던지지 않고 **빈 이력을 반환**합니다.

  ```
  except httpx.HTTPError as e:
      logger.error(...)
      # 실패 시 빈 이력 반환 (추천 서비스는 비핵심 기능)
      return SubscriptionHistoryResponse(userId=user_id, activeProductIds=[])
  ```

  이력이 비면 신규 사용자 취급으로 넘어가 인기순 목록을 돌려줍니다. 즉 **실패가 빈 화면이 아니라 대체 결과**로 이어집니다. 반대로 구독 신청 쪽에서 상품 조회가 실패하면 예외를 던져 신청 자체를 막습니다. 같은 "호출 실패"라도 핵심 거래인지 부가 기능인지에 따라 처리 방식을 다르게 가져간 것입니다.

- **기동 순서에서의 위치** — compose의 `depends_on`에서 recommend-service가 가장 마지막입니다. 추천이 없어도 앞의 흐름은 성립하기 때문입니다.
- **이벤트로 갱신 신호를 받음** — 추천 서비스는 구독 정보를 폴링하지 않고 `subscription.completed` 이벤트를 구독합니다. 학습·갱신 트리거를 이벤트로 받는 구조라, 나중에 모델 재학습을 붙이더라도 발행하는 쪽 코드는 바뀌지 않습니다.

3. 모놀리식이었다면

상황	모놀리식이라면	MSA에서는
추천 규칙을 하루에 두 번 바꾼다	전체 애플리케이션을 다시 빌드·배포. 결제까지 같이 재시작된다.	추천 컨테이너만 교체.
추론 요청이 몰려 CPU를 다 쓴다	같은 프로세스의 로그인·결제 요청도 함께 느려진다.	추천만 스케일 아웃. 다른 서비스 영향 없음.
추천 코드에서 예외가 터진다	스레드·메모리를 공유하므로 전체 장애로 번질 수 있다.	추천 응답만 비고 거래는 계속된다.
파이썬 라이브러리를 써야 한다	JVM 안에서 우회하거나 별도 배치로 분리해야 한다.	서비스 하나를 FastAPI로 만들면 끝.

서비스가 나뉜 대가로 네트워크 호출 실패를 매번 처리해야 하고(`ProductServiceClient`의 try-catch), 이벤트 필드명이 어긋나면 예외 없이 조용히 `null`이 되며(consumer 주석에 그대로 경고가 적혀 있습니다), 커밋 시점까지 신경 써야 합니다(REQUIRES_NEW). **단순한 CRUD만 있는 서비스라면 이 비용을 낼 이유가 없고**, AI 기능처럼 변경 주기·부하·기술 스택·장애 허용도가 다른 요소가 섞여 있을 때 값을 합니다.

Chapter 6. 아키텍처 선택 기준과 이해관계자 가치

1. 모놀리식 / 모듈러 모놀리식 / MSA 비교

기준	모놀리식	모듈러 모놀리식	MSA
배포 단위	애플리케이션 1개	애플리케이션 1개 (내부는 모듈로 분리)	서비스마다 독립 배포
코드 경계	사실상 없음. 어디서든 서로 호출	모듈 간 규약으로 강제. 호출은 인터페이스로만	프로세스와 네트워크로 물리적 강제
데이터	하나의 DB, 조인 자유	하나의 DB지만 스키마·소유를 모듈별로 구분	서비스별 소유. 조인 대신 API 호출·이벤트
통신 비용	메서드 호출 (거의 0)	메서드 호출 (거의 0)	네트워크. 지연·실패를 항상 고려
장애 전파	한 곳이 죽으면 전체 중단	기본적으로 전체 중단	서비스 단위 격리 가능
확장	전체를 통째로 복제	전체를 통째로 복제	필요한 서비스만 확장
조직	한 팀에 적합	모듈별 담당은 가능하나 배포는 함께	팀별 독립 개발·배포
운영 난도	낮음	낮음~중간	높음 (등록소·게이트웨이·브로커·추적 필요)
적합한 때	초기 검증, 소규모, 요구 변동 적음	도메인 경계는 잡고 싶지만 운영 인력이 적을 때	변경 주기·부하·팀이 서로 다른 영역이 섞여 있을 때

2. 판단 체크리스트

세 구조 중 무엇을 고를지는 다음 질문에 몇 개나 "예"가 나오는지로 판단했습니다. 대부분 아니오라면 모놀리식으로 시작하는 편이 낫고, 경계는 필요하지만 운영 인력이 부족하면 모듈러 모놀리식이 절충안이 됩니다.

1. 영역마다 **배포 주기**가 확연히 다른가? (추천은 매일, 결제는 분기에 한 번)
2. 특정 영역만 **부하가 몰리는가**? 그 부분만 늘리고 싶은가?
3. 영역마다 **기술 스택**이 달라야 하는가? (Java + Python)
4. 한 영역의 장애가 다른 영역으로 **번지면 안 되는가**?
5. **팀이 여럿**이고 서로 기다리지 않고 배포해야 하는가?
6. 컨테이너·모니터링·CI/CD를 **감당할 운영 역량**이 있는가?

3. 실제 상황에 적용해 본 판단

상황 A. 이번 실습(2일, 5~6인, 데모가 목표)

솔직하게 판단하면, 기능 규모만 보면 **모듈러 모놀리식으로도 충분한 크기**입니다. 실제로 실습을 하며 컨테이너 열 개를 띄우고 기동 순서와 헬스체크를 기다리는 시간이 기능 개발 시간만큼 들었습니다. 그럼에도 MSA를 택한 근거는 두 가지입니다. 첫째, 목표가 제품 출시가 아니라 **MSA 인프라를 다루는 학습**이라는 것. 둘째, 서비스 경계가 곧 담당 디렉터리가 되어 **5인이 충돌 없이 병렬로 작업**할 수 있다는 것입니다. 이틀이라는 시간에서 병합 충돌로 잃는 시간이 컨테이너 운영 비용보다 컸을 것이라고 봅니다.

상황 B. 팀 아이디어(SK AM)를 실제로 출시한다면

이 경우에는 MSA가 타당합니다. 위 체크리스트에 대입하면 1·2·3·4번이 모두 해당합니다. 카탈로그와 구독은 안정적이지만 추천 규칙은 자주 바뀌고, 추론 부하와 화면 조회 부하의 성격이 다르며, 추천만 파이썬이고, 추천 장애가 구독·결제를 막으면 안 됩니다. 다만 **처음부터 다섯 조각으로 시작할 필요는 없다**고 생각합니다. 회원·카탈로그·구독·결제는 한 덩어리(모듈러 모놀리식)로 두고 **추천만 먼저 떼어내는 방식**이, 초기 팀 규모에서는 가장 이득이 큰 절충안입니다.

상황 C. 사내 관리자 도구를 만든다면

사용자가 수십 명이고 기능도 CRUD 위주라면 모놀리식이 맞습니다. 이 경우 게이트웨이·등록소·브로커는 얻는 것 없이 운영 부담만 늘립니다.

> 세 상황을 놓고 보니 **MSA는 좋은 구조라기보다 값을 치르고 사는 구조**에 가깝습니다. 값을 치를 이유(다른 배포 주기, 다른 부하, 다른 스택, 장애 격리, 여러 팀)가 없으면 사지 않는 편이 낫습니다.

4. 이해관계자별 가치

같은 구조라도 누구에게 어떤 이득인지는 다릅니다. 실습 구조를 이해관계자 관점에서 다시 읽어 보았습니다.

이해관계자	이 구조에서 얻는 가치	근거가 되는 설계 요소	감수해야 하는 것
기획	스프린트마다 동작하는 화면으로 피드백을 받고, 추천 규칙 같은 정책을 자주 바꿔 볼 수 있다	워킹 스켈레톤 단위의 스프린트 분할, 추천 서비스 독립 배포	요구를 서비스 경계에 맞춰 쪼개어 설명해야 한다
개발	담당 서비스 안에서만 작업하므로 충돌이 적고, 다른 서비스가 미완성이어도 API 계약만 있으면 진행할 수 있다	서비스=디렉터리 분리, `internal` API 계약, Eureka 이름 기반 호출	네트워크 실패·이벤트 필드 계약·커밋 시점까지 고려해야 한다
운영	장애가 서비스 단위로 격리되고, 필요한 서비스만 늘릴 수 있다. 로그도 서비스 단위로 본다	컨테이너 분리, healthcheck, 게이트웨이 로깅 필터, 이벤트 발행 성공/실패 로그	구성 요소가 10개로 늘어 기동 순서·모니터링 부담이 커진다
사용자	추천이 죽어도 신청·결제는 되고, 결제 후 상태가 자동으로 바뀌어 기다릴 필요가 없다	추천 실패 시 빈 이력 폴백, `payment.completed` 이벤트 기반 자동 활성화	비동기 처리라 상태 반영이 즉시가 아닐 수 있다

특히 사용자 관점에서 다시 보면, 이벤트 기반 설계는 기술적 선택이면서 동시에 **사용자 경험을 위한 선택**이기도 합니다. 결제 완료 후 활성화를 동기로 처리하면 사용자는 그 시간만큼 화면 앞에서 기다려야 하지만, 이벤트로 처리하면 결제 응답을 먼저 돌려주고 활성화는 뒤에서 진행됩니다.

부록. 궁금했던 점과 직접 알아본 것

---

1. 궁금했던 점 — 게이트웨이가 검증했는데 각 서비스도 또 검증하는 이유

게이트웨이가 JWT를 검증하고 `X-User-Id`로 바꿔 준다면, 뒤에 있는 서비스는 검증할 필요가 없어 보였습니다. 그런데 각 서비스 설정에는 `jwk-set-uri`가 그대로 들어 있고 Resource Server 설정 코드도 (주석 처리되어 있긴 하지만) 준비되어 있습니다. 왜 두 번 검증하는지 궁금했습니다.

찾아본 결과, 이유는 **게이트웨이를 거치지 않는 경로가 실제로 존재하기 때문**이었습니다. compose가 각 서비스 포트를 호스트에 열어 두기 때문에, `localhost:8082`처럼 컨테이너 포트로 직접 호출하면 게이트웨이를 건너뛰고 서비스에 그대로 도달합니다(Chapter 2-5에서 확인). 그리고 현재 각 서비스의 `SecurityConfig`는 `anyRequest().permitAll()`이고, 컨트롤러는 `@RequestHeader("X-User-Id")`를 그대로 신뢰합니다. 두 사실을 합치면, 아래처럼 헤더를 손으로 붙이는 것만으로 다른 사용자 행세가 가능해집니다.

```
curl -H "X-User-Id: 1" http://localhost:8083/api/subscriptions/my
→ 게이트웨이를 거치지 않았는데도 서비스는 1번 사용자로 취급한다
```

찾아보니 이 문제를 막는 방법은 두 가지였습니다. 하나는 **네트워크 차원의 차단**으로, 서비스 포트를 외부에 열지 않고 게이트웨이만 노출하는 방식입니다(실습 compose는 학습·디버깅 편의로 모든 포트를 열어 두었습니다). 다른 하나는 **서비스도 스스로 검증**하는 방식으로, 경계 안쪽도 신뢰하지 않는다는 제로 트러스트 개념입니다. 원래 코드에 `hasAuthority("SCOPE_service.read")`로 내부 경로를 막는 설정이 있었고, 실습에서는 난이도를 조절하려고 이 설정을 주석 처리해 둔 것으로 보입니다. **게이트웨이의 검증은 1차 관문이지 유일한 방어선이 아닙니다.**

2. 직접 알아본 것 — 소스가 없는 이미지의 설정을 바꾸는 방법

실습에서 api-gateway와 auth-server는 소스 없이 이미지(`msa-lecture/api-gateway:1.0`)로만 배포됩니다. 그런데 팀 도메인을 바꾸면서 라우팅 경로를 `/api/courses`에서 `/api/products`로 바꿔야 했습니다. 소스가 없으니 코드를 고칠 수 없는데 어떻게 바꾸는지가 궁금해서 찾아보았습니다.

해법은 Spring Boot의 **설정 파일 탐색 우선순위**였습니다. Spring Boot는 기동할 때 실행 디렉터리의 `./config/application.yml`을 jar 내부 classpath의 설정보다 **먼저** 읽습니다. 이미지의 WorkingDir이 `/app`이므로, 호스트의 설정 파일을 `/app/config/application.yml`로 마운트하면 이미지를 다시 만들지 않고도 라우팅 규칙을 덮어쓸 수 있습니다.

```
volumes:
  - ./api-gateway-config/application.yml:/app/config/application.yml:ro
```

여기에는 함정이 하나 있습니다. `spring.cloud.gateway.routes`는 **리스트라서 부분 병합이 되지 않습니다.** 맵(map) 형태의 설정은 키 단위로 덮어쓰기가 되지만, 리스트는 통째로 교체됩니다. 즉 라우트 한 개만 바꾸고 싶어도 **여섯 개 라우트를 전부 다시 적어야** 하고, 하나라도 빠뜨리면 그 경로는 404가 됩니다.

결국 **코드를 고치는 것 말고도 설정·환경변수·볼륨 마운트라는 층위가 따로 있다**는 것이 이번에 새로 알게 된 부분입니다. 소스가 없는 이미지라고 해서 손댈 방법이 없는 게 아니라, 이미지 바깥에서 설정을 덮어씌우는 방법도 있다는 것을 이번에 처

음 알았습니다.

3. 트러블슈팅 기록 — 재빌드해도 이미지가 갱신되지 않던 문제

항목	내용
증상	코드를 수정한 뒤 도커 이미지를 다시 빌드해 배포해도, 실행 중인 컨테이너가 최신 버전으로 갱신되지 않았습니다. `down → 재빌드 → up` 과정을 여러 번 반복했는데도 변경 사항이 반영되지 않았습니다.
원인	인프라 이미지를 레지스트리에서 직접 받는 과정에서 접속이 몰려 `429 Too Many Requests`가 반복적으로 발생했습니다. 이를 피하려고 강사님이 안내해 주신 우회 방식 — 필요한 이미지를 하나의 tar 파일로 묶어 로컬에서 로드하는 방식 — 으로 배포하고 있었는데, 이 방식이 로컬에 이미 적재된 tar 이미지를 그대로 쓰다 보니 새로 빌드한 이미지가 반영되지 않았습니다.
조치	down·재빌드·up을 반복해도 해결되지 않아 배포 방식 자체를 되돌렸습니다. 우회책 대신 레지스트리에서 이미지를 정상적으로 받는 방식으로 바꾸고, 429를 피하려고 혼잡한 네트워크 대신 회선을 바꿔 이미지를 받았습니다. 이렇게 배포하니 최신 코드가 정상적으로 반영됐습니다.
해결 과정	화면에 나타난 증상만으로는 코드 문제인지 배포 문제인지 구분이 안 돼서, 백엔드·프론트엔드 팀원이 함께 모여 원인을 좁혀 나갔습니다.
재발 방지	배포 우회책을 쓸 때는 그 방식이 이후 재배포에 어떤 부작용을 남기는지 먼저 확인하기로 했습니다. 네트워크 제약(429)은 우회 스크립트를 짜는 것보다 회선을 바꾸는 편이 부작용이 적었습니다.

당시 PO를 맡아 팀 전체 진행도 함께 챙겨야 하는 상황이라, 이 문제를 한두 시간째 붙잡고 있으니 마음이 조급해졌습니다. 백엔드를 맡은 규리님이 원인을 함께 찾아 주었고, 여러 방법을 시도해 본 끝에 핫스팟으로 회선을 바꾸는 방법(네트워크 IP 변경)으로 문제를 해결했습니다. 그 뒤로는 다시 남은 할 일을 쳐내는 데 집중할 수 있었습니다.

이 이슈를 겪으면서 MSA에서는 코드 문제와 배포 문제가 겉보기로 잘 구분되지 않는다는 것과, 원인이 애플리케이션이 아니라 배포 우회책 자체에 있을 수도 있다는 것을 확인했습니다.

4. 마지막으로 느낀 점

수업 중 교수님께서 들어주신 예시에서 궁금한 점이 있었습니다. 승진자 처리를 담당하는 서비스와 유저 서비스가 있다고 할 때, 유저 서비스가 다른 요청을 한창 처리하고 있는 상태에서 승진자 처리 로직이 같은 데이터베이스를 건드려 버리면, 두 서비스가 결국 같은 데이터베이스를 보고 있기 때문에 이렇게 설계하면 안 된다는 말씀이었습니다. 그래서 MSA를 설계하는 기준이 결국 **"데이터베이스"**가 되는 것인지 여쭤봤습니다. 교수님께서는 맞다고, MSA를 설계할 때 데이터베이스를 어떻게 나누고 누가 소유하는지가 매우 중요하다고 답해 주셨습니다.

이론으로 들었을 때는 고개를 끄덕이는 정도였는데, 이번 실습에서 직접 MSA 아키텍처를 만들어 보면서 그 말씀이 실제로 무슨 뜻이었는지 조금씩 이해가 됐습니다. 1장 6절에서 정리했듯, 이 코드 템플릿에서도 테이블 하나에 주인 서비스가 하나씩 정해져 있고, 다른 서비스는 그 테이블을 직접 건드리지 않고 API를 거쳐서만 접근합니다. 교수님이 짚어주신 문제, 즉 여러 서비스가 같은 데이터를 동시에 건드리다 생기는 충돌이 바로 이 구조로 막혀 있는 셈입니다. 말로만 듣던 원칙을 직접 코드로 확인해 본 것이 이번 실습에서 남은 가장 큰 경험이었습니다.